Project Progress Report: Diffusion Text Generation

Date: April 10, 2026

Executive Summary

Our core research question for this project is highly ambitious: *Can we successfully train a discrete diffusion language model end-to-end to generate coherent text, moving away from traditional autoregressive (left-to-right) decoding?* What began as an exploratory proof-of-concept has rapidly evolved into a sophisticated, stable training pipeline. We have successfully pivoted from training from-scratch diffusion models to a more advanced architecture leveraging a pretrained backbone with LoRA adapters. Currently, we have achieved a fully stable, continuous training run on the Rutgers iLab cluster, allowing us to isolate our focus entirely on generation quality and model convergence.

Phase 1: Proof of Concept & Baseline Profiling

Our initial work was housed in the Diffusion_Testing repository, which served as our sandbox for testing from-scratch diffusion models, custom benchmark scripts, and DiffuLLM paths against baseline architectures.

As the scope of our experiments expanded, the repository naturally became highly complex. Different checkpoints required distinct model definitions, and evaluating the models required tracing back specific code versions. To establish a concrete baseline, we conducted a rigorous audit of these early checkpoints. We evaluated them against targeted, short-form prompts (e.g., recipe generation) to test prompt adherence.

The takeaway: While we successfully achieved end-to-end runnable models, the generations exhibited expected early-stage diffusion artifacts—such as token collapse, repetitive punctuation, and semantic drift. This proved that while the mechanics were working, the model needed a stronger foundational prior to generate coherent language.

Phase 2: Architectural Pivot (**Diffusion_Testing2**)

Rather than patching the original from-scratch models, we executed a strategic pivot. We deprecated the initial workspace and engineered a clean, streamlined environment: Diffusion_Testing2.

This wasn't just a code cleanup; it was a fundamental shift in our ML architecture. We integrated a **pretrained Qwen backbone paired with LoRA adapters and a masked diffusion decoding process.** By anchoring the diffusion objective to a highly capable language model, we provided the system with the linguistic foundation necessary to support complex decoding.

During this phase, we also overhauled our evaluation pipeline. We engineered a robust prompt-isolation testing method that perfectly preserves the prompt and strictly captures the generated completion, giving us an accurate, noiseless metric for how well the model follows instructions.

Phase 3: Infrastructure, SLURM, and iLab Deployment

With the architecture locked in, our next major hurdle was scaling the training process. Deploying the Qwen-based diffusion model to the Rutgers iLab required significant infrastructure debugging.

We successfully resolved multiple HPC bottlenecks:

- Overhauled SLURM job configurations for optimal resource allocation.
- Debugged and bypassed unreliable virtual environments by engineering a robust Conda environment specifically tailored to the cluster.
- Stabilized the data pipeline to handle prolonged, uninterrupted training.

The result is our biggest technical milestone to date: We have a fully stable, continuous training job currently running on the iLab cluster, successfully passing step_6500.

Current Snapshot & Next Steps

Our ML pipeline and infrastructure are now fully solved, allowing us to focus entirely on generation tuning.

- **Infrastructure Locked:** The pipeline on Rutgers iLab is highly stable, reliably generating and saving checkpoints (currently at step_6500+).
- **Evaluation Streamlined:** We successfully distilled the latest iLab checkpoint into a smaller, inference-only model for rapid local testing.
- **Generation Quality (The Final Hurdle):** When comparing the new Qwen-backed checkpoints against our Phase 1 baselines, we are seeing a much stronger structural awareness of the prompt. While the model still struggles with token stability and occasional filler repetition rather than outputting a clean recipe, the architectural foundation is fundamentally sound.

Next Steps: Our immediate focus is diagnosing the decoding instability in the latest checkpoints and fine-tuning the masking strategy to push the output from "structurally aware" to "semantically coherent."